Лабораторная работа №2

Калькулятор. Программирование логики на JavaScript

1. Цель работы

Освоить программирование логики и интерактивности веб-страниц с помощью JavaScript: получить доступ к HTML-элементам из скрипта, настроить обработчики событий и реализовать работающую логику калькулятора, вёрстка которого была выполнена в ЛР №1 (HTML/CSS в стилистике discord.com).

2. Подключение JavaScript к странице

Код вынесен во внешний файл script.js и подключён в <head> документа через тег <script> с атрибутом src — это более организованный подход, чем встроенный скрипт прямо в HTML:

```
<head>

  <meta charset="UTF-8">

  <title>Калькулятор — Discord style</title>

  <link rel="stylesheet" href="style.css">

  <script type="text/javascript" src="script.js"></script>

</head>
```

3. Доступ к HTML-элементам из JavaScript

Для доступа к элементам использованы два метода:

- document.getElementById(id) — для одиночных элементов: поля вывода (#result) и каждой из кнопок операций (btn_op_plus, btn_op_minus, btn_op_mult, btn_op_div, btn_op_clear, btn_op_equal, btn_op_sign, btn_op_percent);
- document.querySelectorAll('[id^="btn_digit_"]') — чтобы одним вызовом получить сразу все цифровые кнопки (0-9 и точку), id которых начинается с btn_digit_, вместо того чтобы перечислять 11 id вручную.

Обработчики событий назначены через свойство элемента onclick (а не через inline-атрибут в HTML или addEventListener) — такой способ был выбран, так как на каждую кнопку нужен ровно один обработчик, и это делает код компактнее. Ниже перечислены события, которые в принципе существуют в DOM, и то, как они соотносятся с задачей:

Событие	**Когда происходит**	**Где применимо в калькуляторе**

click	клик левой кнопкой мыши по элементу	нажатие на кнопку цифры/операции

DOMContentLoaded/load	HTML загружен и DOM построен	весь код калькулятора обёрнут в window.onload

keydown/keyup	нажатие/отпускание клавиши клавиатуры	не используется в этой ЛР

mouseover / mouseout	наведение / уход курсора с элемента	в CSS реализовано через :hover

4. Логика калькулятора — разбор кода

4.1. Переменные состояния

Весь код обёрнут в window.onload, чтобы скрипт выполнялся только после полной загрузки DOM. Четыре переменные хранят текущее состояние калькулятора:

```
let a = '';                     // первое число (строка, набирается по цифрам)
let b = '';                     // второе число
let expressionResult = '';      // результат вычисления
let selectedOperation = null;   // выбранная операция (+, −, x, /)
```

4.2. Ввод цифр

Функция onDigitButtonClicked дописывает нажатую цифру в конец строки a или b (в зависимости от того, выбрана ли уже операция), а также следит, чтобы в числе не оказалось двух точек:

```
function onDigitButtonClicked(digit) {

  if (!selectedOperation) {

    if ((digit !== '.') || (digit === '.' && !a.includes(digit))) {

      a += digit;

    }

    outputElement.innerHTML = a;

  } else {

    if ((digit !== '.') || (digit === '.' && !b.includes(digit))) {

      b += digit;

      outputElement.innerHTML = b;

    }

  }

}
```

Эта функция вызывается в обработчике клика каждой из цифровых кнопок (цикл forEach по результату querySelectorAll), текст кнопки (button.innerHTML) передаётся как нажатая цифра:

```
digitButtons.forEach(button => {

  button.onclick = function () {

    const digitValue = button.innerHTML;

    onDigitButtonClicked(digitValue);

  };

});
```

4.3. Операции, очистка и вычисление результата

Кнопки +, -, x, / просто запоминают выбранную операцию в selectedOperation (и дальше ввод цифр идёт уже во вторую переменную b). Кнопка C сбрасывает все переменные в исходное состояние. Кнопка = проверяет, что оба числа и операция заданы, и вычисляет результат через switch по selectedOperation:

```
switch (selectedOperation) {

  case 'x': expressionResult = (+a) * (+b); break;
  case '+': expressionResult = (+a) + (+b); break;

  case '-': expressionResult = (+a) - (+b); break;

  case '/': expressionResult = (+a) / (+b); break;

  default: break;

}

a = expressionResult.toString();  // результат становится новым a —

b = '';                           // так можно сразу продолжать счёт

selectedOperation = null;

outputElement.innerHTML = a;
```

Унарный плюс (+a, +b) приводит строку к числу перед арифметикой — без этого, например, '2' + '3' дало бы строку '23', а не число 5.

4.4. Дополнительно реализованные операции

Помимо базового плана ЛР, в script.js дополнительно запрограммированы две кнопки, которые в вёрстке ЛР №1 были нерабочими:

- +/- — меняет знак текущего набираемого числа (a, если операция ещё не выбрана, иначе b) через проверку a.startsWith('-') и добавление/удаление минуса в начале строки;
- % — делит текущее набираемое число на 100, как в большинстве настольных калькуляторов.

```
document.getElementById('btn_op_sign').onclick = function () {
  if (!selectedOperation) {
    if (a === '') return;
    a = a.startsWith('-') ? a.slice(1) : '-' + a;
    outputElement.innerHTML = a;
  } else {
    if (b === '') return;
    b = b.startsWith('-') ? b.slice(1) : '-' + b;
    outputElement.innerHTML = b;
  }
};
```

5. Проверка работоспособности

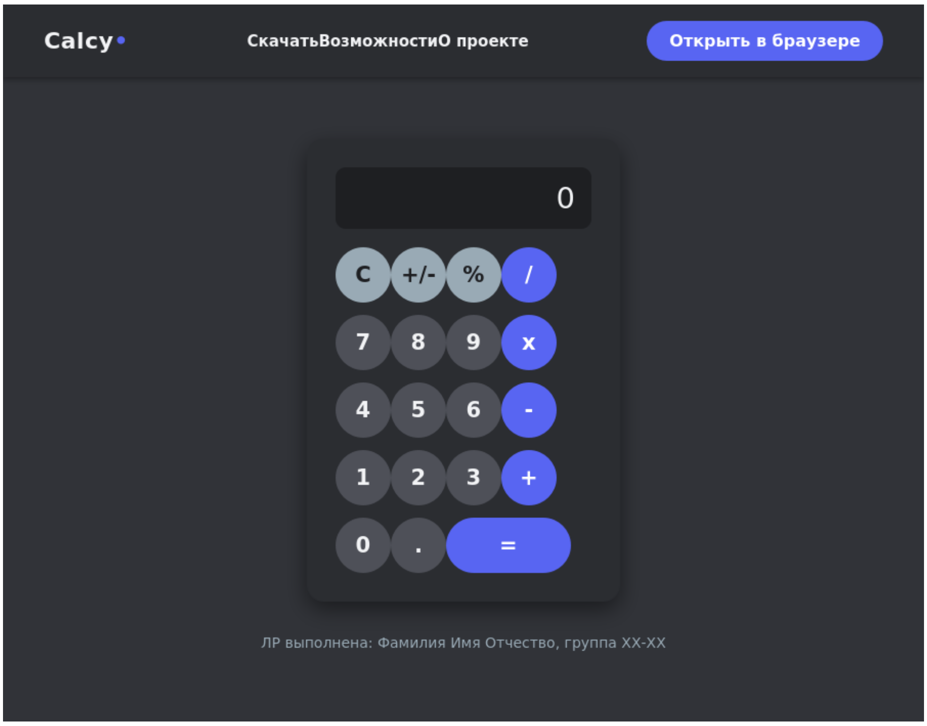
Логика проверена автоматическими тестами (через jsdom — эмуляция DOM в Node.js: загружается calculator.html, выполняется script.js, после чего программно вызываются обработчики onclick нужных кнопок и сверяется содержимое #result). Результаты проверки:

```
12 + 8 =        20      OK

9 x 7 =         63      OK
```

```
sign(5) =       -5      OK

percent(50) =   0.5     OK

7 / 2 =         3.5     OK

(10-3)+1 =      8       OK   (продолжение счёта после "=")
```

Все проверенные сценарии — сложение, умножение, деление, смена знака, процент и продолжение вычислений после нажатия «=» — отработали корректно.

Внешний вид страницы после подключения script.js (визуально не изменился относительно ЛР №1, так как JS влияет только на поведение, не на разметку/стили):



ЛР выполнена: Фамилия Имя Отчество, группа XX-XX

6. Запуск через Live Server

Для просмотра калькулятора в работе страница запускалась через расширение Live Server в VS Code:

- В VS Code открыта папка проекта (calculator.html, style.css, script.js в одной директории);
- Установлено расширение Live Server (панель Extensions → поиск «Live Server» → Install);
- По клику правой кнопкой мыши на calculator.html выбран пункт «Open with Live Server» (либо кнопка «Go Live» в нижней панели);
- Страница открылась в браузере по адресу вида http://127.0.0.1:5500/calculator.html — в отличие от прямого открытия файла (file:///...), Live Server поднимает локальный веб-сервер и автоматически перезагружает страницу при сохранении изменений в коде.

7. Задания для самостоятельной проработки — статус

Из предложенного списка дополнительных заданий в данной работе реализованы:

- 1) смена знака +/- — реализовано (см. п. 4.4);
- 2) вычисление процента % — реализовано (см. п. 4.4).

Остальные пункты (backspace, смена цвета фона/окна вывода по кнопке, √, x^2, x!, тройной ноль, накапливаемые сложение/вычитание, индивидуальная операция) можно реализовать по аналогии: добавить новый обработчик onclick на соответствующую кнопку в script.js, используя переменные a, b, selectedOperation и outputElement так же, как в уже реализованных функциях.